Factors

Data Structures

- vectors
- *`factors`*
- lists
- matrices
- arrays
- dataframes

Factors

Factors are a peculiar data type: although they look like simple `characters`, they are actually a type of `integer` vector with some additional properties.

Creating a factor

Factors can be created either by converting a character vector with the `as.factor()` command …

```
1 char_vect <- rep(c("hello","ciao", "hola"), each = 2)
2 char_vect
```

```
[1] "hello" "hello" "ciao"  "ciao"  "hola"  "hola"
```

```
1 my_fact <- as.factor(char_vect)
2 my_fact
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: ciao hello hola
```

Creating a factor

… or by explicitly creating a factor with the **`factor()`** command:

```
my_fact <- factor(x = char_vect)
my_fact
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: ciao hello hola
```

… equivalently

```
my_fact <- factor(x = rep(c("hello","ciao", "hola"), each = 2))

my_fact <- factor(x = c("hello", "hello", "ciao",
                        "ciao", "hola", "hola"))
```

Properties of factors

In practice, they assign a label to an integer value:

```
1 # typeof determines the type or storage mode of any object
2 typeof(my_fact)
```

```
[1] "integer"
```

Why these numbers?

```
1 as.integer(my_fact)
```

```
[1] 2 2 1 1 3 3
```

```
1 my_fact
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: ciao hello hola
```

Factors store `levels()` as metadata,

```
levels(my_fact)
```

```
[1] "ciao"  "hello" "hola"
```

regardless of which ones are actually present in the vector. For example, if I create a factor made only of the elements of `my_fact` different from *ciao*:

```
my_fact2 <- my_fact[my_fact!="ciao"]
```

The **levels** of `my_fact2` will be the **same** as those of `my_fact` (*ciao* included), even though *ciao* is **no longer present as an observation**:

```
my_fact2
```

```
[1] hello hello hola  hola
Levels: ciao hello hola
```

```
levels(my_fact2)
```

```
[1] "ciao"  "hello" "hola"
```

However, we can drop the levels that are no longer needed with the `droplevels()` command:

```
# my_fact2 without the levels that have no observations
droplevels(my_fact2)
```

```
[1] hello hello hola  hola
Levels: hello hola
```

```
# modify my_fact2 by removing the unused levels
my_fact2 <- droplevels(my_fact2)
my_fact2
```

```
[1] hello hello hola  hola
Levels: hello hola
```

Reorder levels

Why reorder? The **first level** is the **reference category**: statistical models often compare every group to it.

```r
# the group of each participant in an experiment
group <- factor(x = c("placebo", "caffeine", "nicotine",
                      "placebo", "caffeine"))
group
```

```
[1] placebo  caffeine nicotine placebo  caffeine
Levels: caffeine nicotine placebo
```

```r
levels(group)       # the table of levels
```

```
[1] "caffeine" "nicotine" "placebo"
```

```r
as.integer(group)   # the codes saved in the factor
```

```
[1] 3 1 2 3 1
```

Here `placebo` should be the reference, but the alphabetical order puts it **last**.

Reorder: placebo first

`levels` in `factor()`: the levels **in the order we want**

```
group_ord <- factor(group,
  levels = c("placebo", "caffeine", "nicotine"))

levels(group_ord)
```

```
[1] "placebo"  "caffeine" "nicotine"
```

```
as.integer(group_ord)    # new codes...
```

```
[1] 1 2 3 1 2
```

```
group_ord
```

```
[1] placebo  caffeine nicotine placebo  caffeine
Levels: placebo caffeine nicotine
```

```
group
```

```
[1] placebo  caffeine nicotine placebo  caffeine
Levels: caffeine nicotine placebo
```

A shortcut: `relevel()`

If we only need to choose the **reference category**, `relevel()` moves it to the first place and leaves the others as they are:

```
1 levels(group)
```

```
[1] "caffeine" "nicotine" "placebo"
```

```
1 group_ref <- relevel(group, ref = "placebo")
2 levels(group_ref)
```

```
[1] "placebo"  "caffeine" "nicotine"
```

Rename: `levels(x) <- ...`

The new names are assigned **by position**: the first name goes to the first level, and so on.

```
levels(group_ord)        # ALWAYS check the current order first!
```

```
[1] "placebo"  "caffeine" "nicotine"
```

```
group_short <- group_ord
levels(group_short) <- c("PLA", "CAF", "NIC")
group_ord
```

```
[1] placebo  caffeine nicotine placebo  caffeine
Levels: placebo caffeine nicotine
```

```
group_short
```

```
[1] PLA CAF NIC PLA CAF
Levels: PLA CAF NIC
```

⚠️ The trap: renaming in the wrong order

We wanted placebo first, so we write the names in the order we want…

```r
group_wrong <- group
levels(group_wrong) <- c("placebo", "caffeine", "nicotine")
group
```

```
[1] placebo  caffeine nicotine placebo  caffeine
Levels: caffeine nicotine placebo
```

```r
group_wrong
```

```
[1] nicotine placebo  caffeine nicotine placebo
Levels: placebo caffeine nicotine
```

No error, no warning, but every value is now wrong! `levels(x) <-` **renames**, it does not reorder: “caffeine” (first level) was renamed “placebo”, and so on.

The safe way: `factor(x, levels, labels)`

To reorder **and** rename in one step:

```
group_new <- factor(group,
  levels = c("placebo", "caffeine", "nicotine"),
  labels = c("PLA", "CAF", "NIC"))
group
```

```
[1] placebo  caffeine nicotine placebo  caffeine
Levels: caffeine nicotine placebo
```

```
group_new
```

```
[1] PLA CAF NIC PLA CAF
Levels: PLA CAF NIC
```

- `levels`: the **existing** values, in the order we want
- `labels`: their **new** names, in the same order: matched **by name**, so the data cannot get mixed up

Summary

to…	use	does the data change?
see the levels	`levels(x)`	no
reorder the levels	`factor(x, levels = c(...))`	no
choose the reference	`relevel(x, ref = "...")`	no
rename the levels	`levels(x) <- c(...)`	no, **if you follow the current order**
reorder + rename	`factor(x, levels = c(...), labels = c(...))`	no
remove unused levels	`droplevels(x)`	no

Order of the levels ≠ ordered factor

The order of the levels only decides **which comes first** (reference, tables, plots). It does not mean that one group is “bigger” than another.

Ordinal Factors

A subtype of factors are ***ordered factors***, which correspond to ordinal variables, i.e. variables whose levels are sorted in increasing order:

```
my_ord_fac <- as.ordered(my_fact)

my_ord_fac # alphabetical order
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: ciao < hello < hola
```

```
attributes(my_ord_fac)
```

```
$levels
[1] "ciao"  "hello" "hola"

$class
[1] "ordered" "factor"
```

```r
my_ord_fac <- factor(
  x = c("never", "sometimes", "often", "always"),
  ordered = TRUE)
my_ord_fac# alphabetical order
```

```
[1] never     sometimes often     always
Levels: always < never < often < sometimes
```

So what should we do?

```r
my_ord_fac <- factor(
  x = c("never", "sometimes", "often", "always"),
  levels = c("never", "sometimes", "often", "always"),
  ordered = TRUE)
my_ord_fac
```

```
[1] never     sometimes often     always
Levels: never < sometimes < often < always
```

Let’s practice!

Open this link and keep it open:

https://etherpad.wikimedia.org/p/arca-corsoR